МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 3

Тема: Структурные паттерны: Адаптер, Мост

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы структурных паттернов проектирования «Адаптер» (Adapter) и «Мост» (Bridge) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два структурных паттерна из списка: «Адаптер», «Мост».

В соответствии с заданием были выбраны паттерны «Адаптер» (Adapter) и «Мост» (Bridge).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, заданный последовательностью выбранных ячеек, а также демонстрирует работу структурных паттернов: адаптер используется для преобразования последовательности в строковый формат, а мост - для независимого изменения способа визуализации выбранных ячеек.

1. Описание паттерна «Адаптер»

Паттерн «Адаптер» (Adapter) - это структурный паттерн проектирования, который позволяет объектам с несовместимыми интерфейсами работать вместе. Адаптер выступает в роли «переходника» между двумя классами: он реализует интерфейс, ожидаемый клиентом, и делегирует вызовы адаптируемому объекту, преобразуя данные по необходимости.

Данный паттерн применяется в ситуациях, когда требуется использовать существующий класс, интерфейс которого не соответствует ожиданиям остальной системы, и при этом нельзя или нецелесообразно изменять сам класс.

В разработанном приложении паттерн «Адаптер» используется для того, чтобы представить внутреннюю последовательность графического пароля (список целых чисел) в виде строки с разделителями и обратно.

В программе определены следующие роли:

– SequenceTarget - целевой интерфейс, который ожидает клиентский код. Он определяет методы get_sequence_string() и set_sequence_from_string(data);
– ListAdaptee - адаптируемый класс, который хранит последовательность в виде списка чисел [1, 5, 9, 8];
– SequenceAdapter - адаптер, который реализует SequenceTarget и преобразует список в строку вида "1-5-9-8" и обратно.

Таким образом, основной код приложения может работать со строковым представлением последовательности, не зная о внутреннем формате хранения.

2. Описание паттерна «Мост»

Паттерн «Мост» (Bridge) - это структурный паттерн проектирования, который разделяет абстракцию и её реализацию так, чтобы они могли изменяться независимо друг от друга. Вместо наследования от одной иерархии классов паттерн использует композицию: абстракция содержит ссылку на объект реализации и делегирует ему часть работы.

Данный паттерн применяется в ситуациях, когда у класса есть несколько независимых измерений изменчивости, и наследование привело бы к взрывному росту числа подклассов.

В разработанном приложении паттерн «Мост» используется для разделения:

– абстракции - класса AppWindow, который управляет интерфейсом, обрабатывает выбор ячеек и хранит последовательность пароля;
– реализации - интерфейса GridRenderer, который отвечает за визуальное отображение связей между выбранными ячейками.

На основе GridRenderer реализованы две конкретные реализации:

– SimpleRenderer - простая реализация, которая не рисует линии между ячейками (используются только подсвеченные кнопки);
– LineRenderer - продвинутая реализация, которая рисует линии со стрелками между выбранными ячейками.

Абстракция AppWindow содержит ссылку на GridRenderer и делегирует ему отрисовку связей. Пользователь может переключать реализацию в процессе работы приложения через кнопки «Простой» и «С линиями».

Таким образом, логика работы с паролем и способ его визуализации изменяются независимо друг от друга.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter для создания графического интерфейса пользователя.

Основное окно приложения содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются следующие требования:

- длина последовательности составляет от 4 до 9 ячеек;
- одна и та же ячейка не может быть выбрана повторно;
- каждая выбранная ячейка отображается в текущей последовательности;
- созданный пароль сохраняется в виде SHA-256 хэша для последующей проверки.

Приложение содержит два основных режима:

- Создание пароля - пользователь выбирает последовательность ячеек и сохраняет её;
- Вход - пользователь вводит ранее установленную последовательность и выполняет проверку.

Приложение позволяет:

- выбирать отдельные ячейки;
- формировать последовательность графического пароля;
- очищать выбранную последовательность;
- переключаться между режимами «Вход» и «Создать пароль»;

– переключать способ отрисовки связей между ячейками (паттерн «Мост»);
– просматривать строковое представление последовательности (паттерн «Адаптер»);
– сбрасывать сохранённый пароль.

Для улучшения внешнего вида кнопки и ячейки сетки реализованы в виде скруглённых прямоугольников на основе Canvas.

4. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

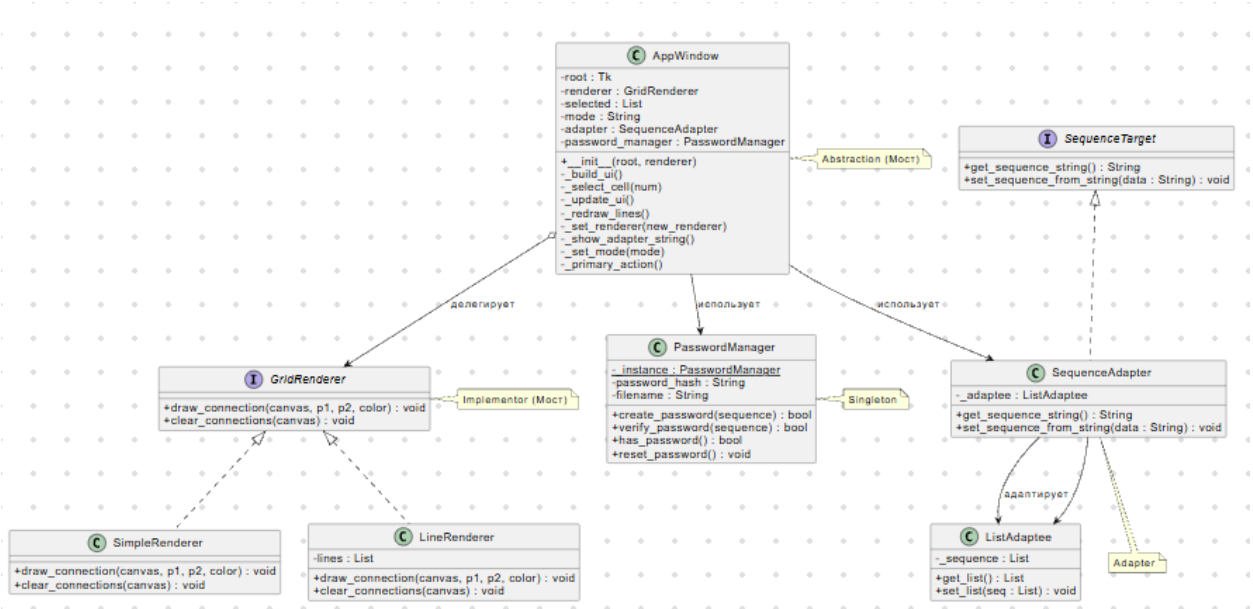


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит основные классы и интерфейсы приложения: AppWindow, SequenceTarget, ListAdaptee, SequenceAdapter, GridRenderer, SimpleRenderer, LineRenderer, а также вспомогательный класс PasswordManager, реализующий паттерн «Одиночка» (из предыдущей лабораторной работы).

Интерфейс SequenceTarget и классы ListAdaptee и SequenceAdapter реализуют паттерн «Адаптер».

Интерфейс GridRenderer, его реализации SimpleRenderer и LineRenderer, а также абстракция AppWindow реализуют паттерн «Мост».

Класс AppWindow является основным классом приложения и использует адаптер для работы со строковым представлением последовательности и мост для делегирования отрисовки.

Класс PasswordManager реализует паттерн «Одиночка» и отвечает за сохранение и проверку хэша пароля.

5. Реализация паттерна «Адаптер»

Основная часть реализации паттерна «Адаптер» находится в классах SequenceTarget, ListAdaptee и SequenceAdapter.

Целевой интерфейс:

```
class SequenceTarget(ABC):
    @abstractmethod
    def get_sequence_string(self) -> str:
        pass

    @abstractmethod
    def set_sequence_from_string(self, data: str):
        pass
```

Адаптируемый класс - существующий список чисел:

```
class ListAdaptee:
    def __init__(self, sequence: list):
        self._sequence = sequence

    def get_list(self) -> list:
        return self._sequence

    def set_list(self, seq: list):
        self._sequence = seq
```

Адаптер:

```
class SequenceAdapter(SequenceTarget):
    def __init__(self, adaptee: ListAdaptee):
```

```
        self._adaptee = adaptee

    def get_sequence_string(self) -> str:
        return "-".join(map(str, self._adaptee.get_list()))

    def set_sequence_from_string(self, data: str):
        if not data:
            self._adaptee.set_list([])
            return
        parts = data.split("-")
        self._adaptee.set_list([int(p) for p in parts])
```

В AppWindow адаптер используется следующим образом: при выборе ячейки список обновляется и передаётся адаптеру через set_list, а кнопка «Показать строку (Адаптер)» вызывает get_sequence_string() и показывает пользователю, как список [1, 5, 9, 8] превращается в строку "1-5-9-8".

Таким образом, клиентский код работает через SequenceTarget, не зная о внутреннем устройстве ListAdaptee.

6. Реализация паттерна «Мост»

Основная часть реализации паттерна «Мост» находится в интерфейсе GridRenderer и его реализациях SimpleRenderer и LineRenderer.

Интерфейс реализации:

```
class GridRenderer(ABC):
    @abstractmethod
    def draw_connection(self, canvas, p1, p2, color):
        pass

    @abstractmethod
    def clear_connections(self, canvas):
        pass
```

Простая реализация - ничего не рисует:

```
class SimpleRenderer(GridRenderer):
    def draw_connection(self, canvas, p1, p2, color):
        pass

    def clear_connections(self, canvas):
        pass
```

Продвинутая реализация - рисует линии между выбранными ячейками:

```
class LineRenderer(GridRenderer):
    def __init__(self):
        self.lines = []

    def draw_connection(self, canvas, p1, p2, color):
        line = canvas.create_line(p1[0], p1[1], p2[0], p2[1],
                                  fill=color, width=4, arrow=tk.LAST)
        self.lines.append(line)

    def clear_connections(self, canvas):
        for line in self.lines:
            canvas.delete(line)
        self.lines.clear()
```

Абстракция AppWindow хранит ссылку на реализацию и делегирует ей отрисовку:

```
class AppWindow:
    def __init__(self, root, renderer: GridRenderer):
        self.renderer = renderer
        ...
```

Переключение реализации в процессе работы:

```
def _set_renderer(self, new_renderer):
    self.renderer.clear_connections(self.canvas)
    self.renderer = new_renderer
```

self._redraw_lines()

Таким образом, абстракция AppWindow и реализации GridRenderer изменяются независимо: можно добавить новую реализацию (например, AnimatedRenderer) без изменения AppWindow.

7. Результаты работы программы

После запуска приложения пользователю отображается главное окно с сеткой из девяти скруглённых ячеек.

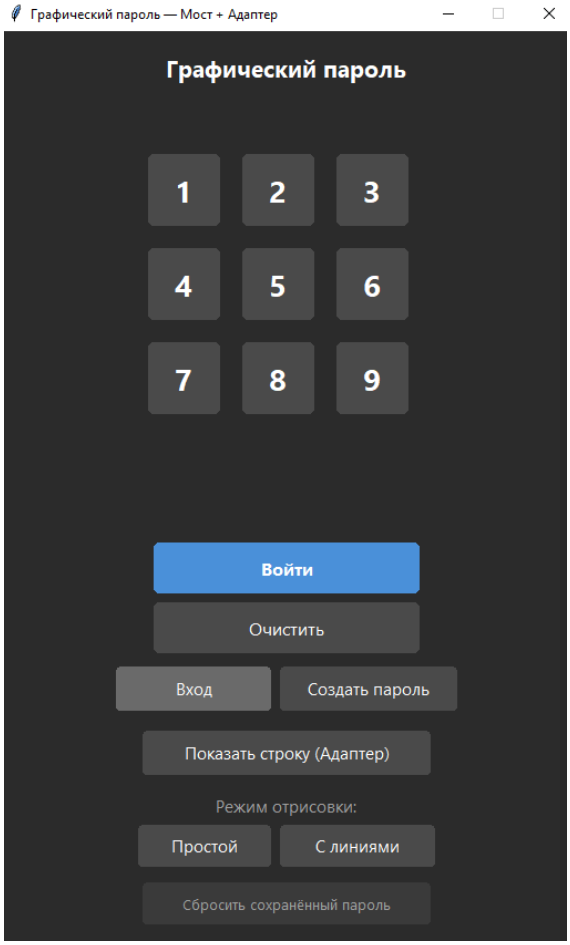


Рисунок 2 - Главное окно приложения

Пользователь может выбирать ячейки и формировать последовательность графического пароля. При выборе ячейки она подсвечивается, а последовательность отображается под сеткой.

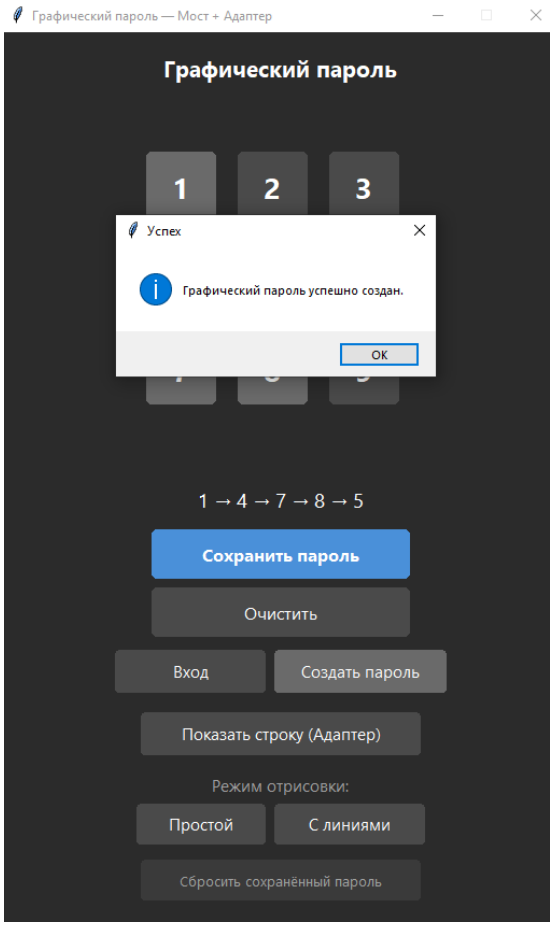


Рисунок 3 - Формирование последовательности графического пароля

При нажатии кнопки «Показать строку (Адаптер)» появляется диалоговое окно с демонстрацией работы адаптера: внутренний список [1, 5, 9, 8] превращается в строку "1-5-9-8".

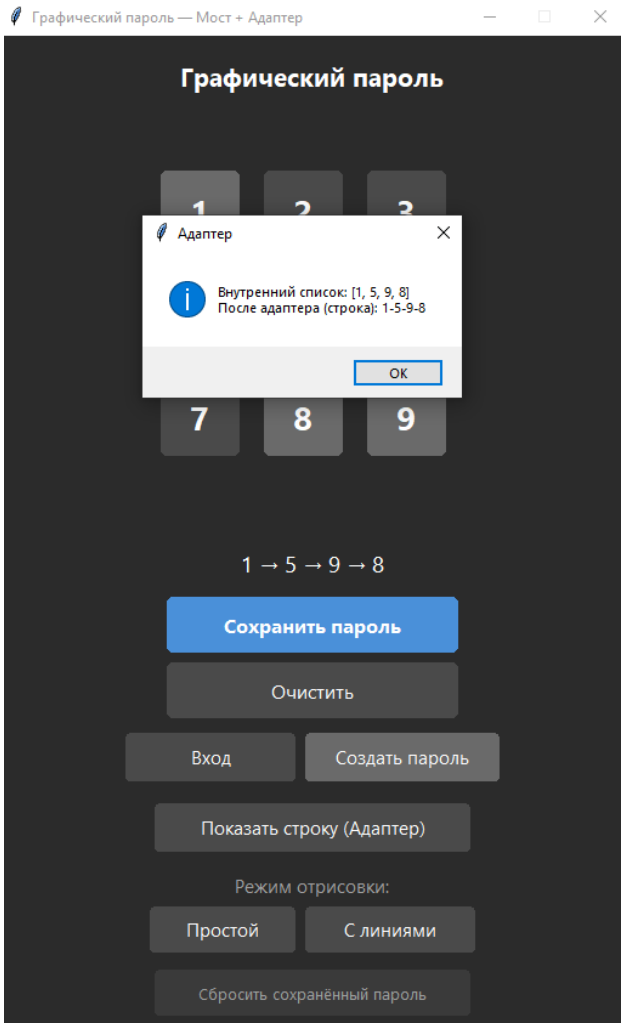


Рисунок 4 - Демонстрация работы паттерна «Адаптер»

Для демонстрации паттерна «Мост» пользователь может переключать режим отрисовки:

– Простой - связи между выбранными ячейками не рисуются.
– С линиями - между выбранными ячейками рисуются линии со стрелками (аналогично графическому ключу в мобильных ОС).

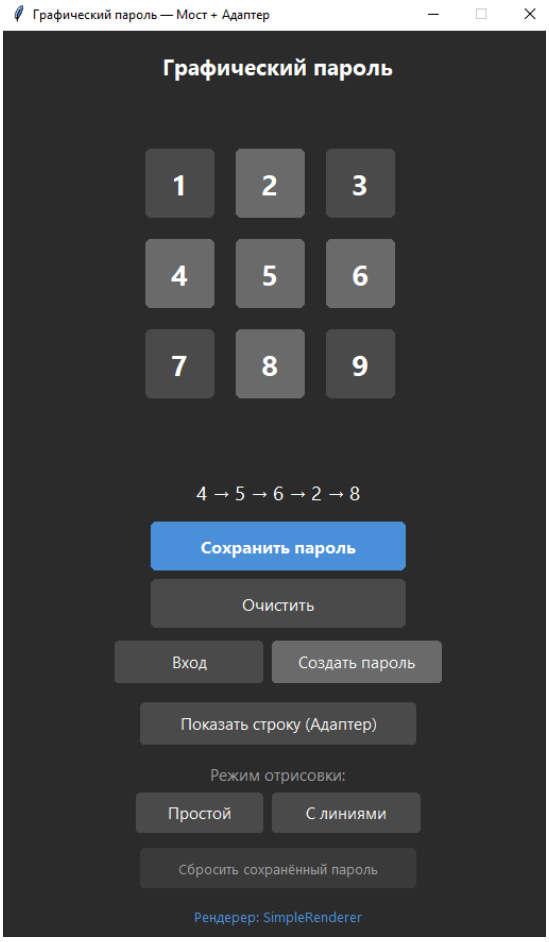


Рисунок 5 - Режим «Простой»



Рисунок 6 - Режим «С линиями» (демонстрация паттерна «Мост»)

Для сохранения пароля пользователь переключается в режим «Создать пароль», выбирает последовательность и нажимает кнопку «Сохранить пароль». При успешном сохранении выводится уведомление.

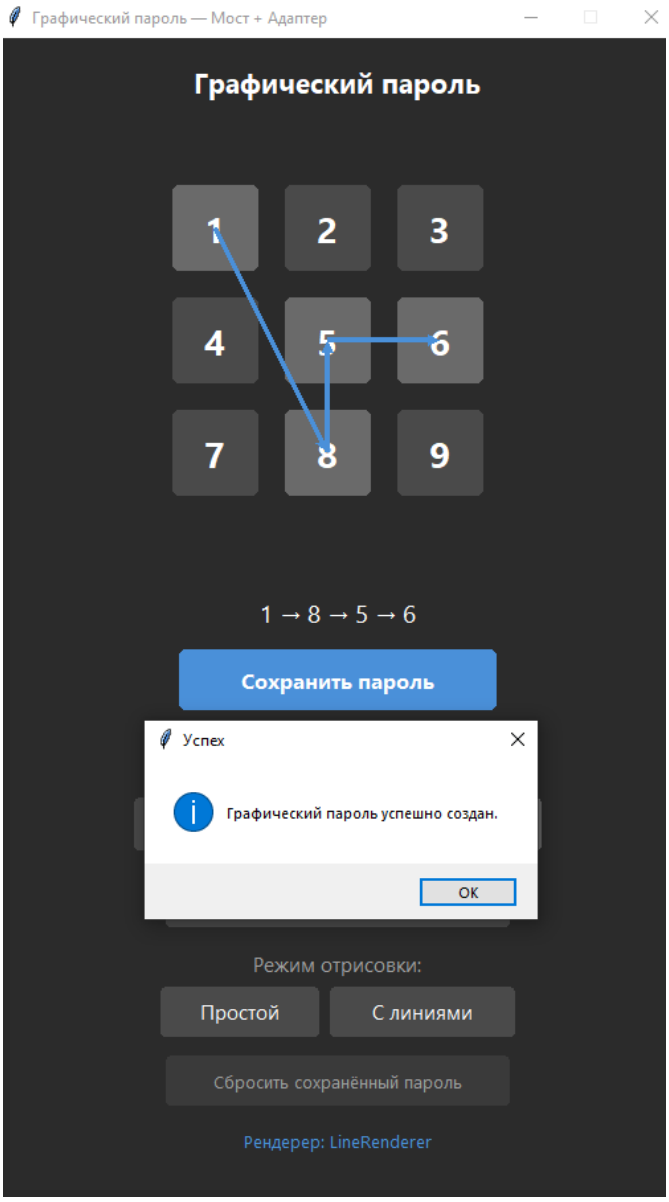


Рисунок 7 - Создание графического пароля

Для проверки пароля пользователь переключается в режим «Вход», вводит последовательность и нажимает кнопку «Войти». При правильном вводе выводится сообщение «Пароль верный. Доступ разрешён.», при неправильном - «Пароль неверный. Доступ запрещён.».

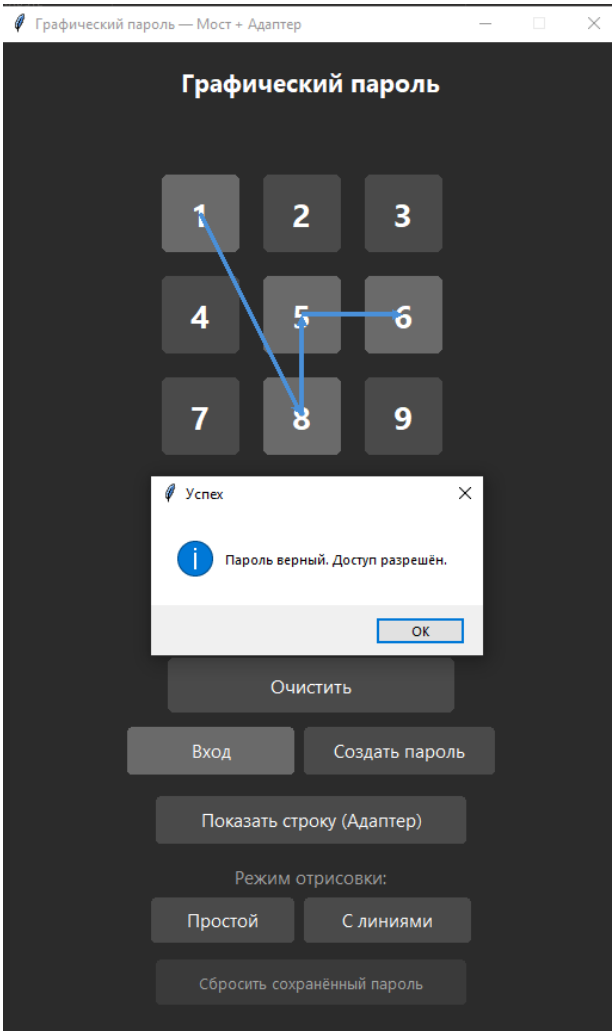


Рисунок 8 - Проверка пароля

Также в приложении предусмотрена возможность удаления сохранённого пароля.

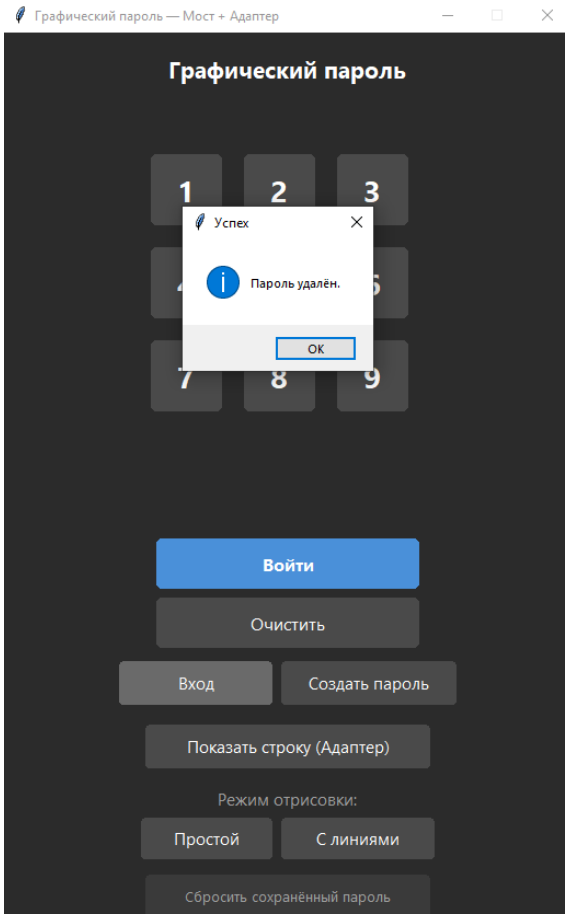


Рисунок 9 - Сброс сохранённого пароля

8. Исходный код

Приложение разработано на языке программирования Python. Для создания графического интерфейса используется библиотека Tkinter.

Основные элементы программы:

- AppWindow - основной класс графического интерфейса (Abstraction в паттерне «Мост»);
- SequenceTarget - целевой интерфейс адаптера;
- ListAdaptee - адаптируемый класс (список чисел);
- SequenceAdapter - адаптер, преобразующий список в строку и обратно;
- GridRenderer - интерфейс реализации (Implementor в паттерне «Мост»);
- SimpleRenderer - реализация без отрисовки связей;
- LineRenderer - реализация с отрисовкой связей;

- PasswordManager - класс управления паролем, реализующий Singleton (из лабораторной работы №1);
- validate_sequence() - вспомогательная функция проверки корректности последовательности;
- hash_sequence() - вспомогательная функция получения SHA-256 хэша;
- RoundedButton, rounded_rect() - вспомогательные элементы для создания скруглённых кнопок и ячеек;
- graphical_password.json - файл хранения хэша пароля.

Полный исходный код программы представлен в приложении к лабораторной работе. Код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два структурных паттерна проектирования: «Адаптер» (Adapter) и «Мост» (Bridge). Паттерн «Адаптер» применён для преобразования внутренней последовательности пароля (списка чисел) в строковый формат и обратно через классы SequenceTarget, ListAdaptee и SequenceAdapter. Паттерн «Мост» применён для разделения абстракции AppWindow и реализации GridRenderer, что позволяет независимо изменять способ визуализации связей между выбранными ячейками (SimpleRenderer и LineRenderer). В результате получена работоспособная программа, демонстрирующая практическое применение структурных паттернов при разработке прикладного программного обеспечения.